DOCUMENTO DE ARQUITECTURA

Emiralia

Mapa completo de la arquitectura, interconexiones, decisiones de diseno y roadmap de escalabilidad

AI-Native PropTech Platform

Documento confidencial · 10 de marzo de 2026 · v1.0

Indice

01 Diagrama General de Infraestructura

02 Aplicaciones (Website, API, Dashboard)

03 Tools y Scripts

04 Base de Datos (PostgreSQL)

05 Infraestructura Docker

06 Framework WAT (.claude/)

07 Interconexiones entre Documentos

08 Cadenas de Skills y Workflows

09 Flujos de Datos End-to-End

10 Decisiones de Arquitectura

11 Mejoras Futuras para Escalar

12 Patrones Arquitectonicos

01. Diagrama General de Infraestructura

Emiralia esta compuesta por **3 aplicaciones**, **5 servicios Docker**, y un ecosistema de **30+ tools** coordinados por el framework WAT.

```
                        ┌──────────────────┐
                        │   Telegram Bot   │
                        │  tools/telegram/ │
                        └──────────┬───────┘
                                   │ Ideas / Chat
                                   ▼
┌──────────────┐   REST    ┌──────────────┐   SQL    ┌──────────────────┐
│   Website    │◄──────────►│   API        │◄─────────►│   PostgreSQL 16  │
│  (Publico)   │  :3001    │  Express     │  :5433   │                  │
│  Vite + TW   │           │  apps/api/   │          │  17 tablas       │
│  apps/website│           └──────────────┘          └──────────┬───────┘
└──────────────┘                                                 │
                                                                 │
┌──────────────┐  SQL+API  ┌──────────────┐                     │
│  Dashboard   │◄──────────►│  Dashboard   │◄────────────────────┘
│  React 19    │   :5173   │  Server      │
│  Vite + RR7  │           │  Express 5   │  :3001
└──────────────┘           └──────┬───────┘
                                  │ imports
                                  ▼
                       ┌──────────────────────┐
                       │   tools/pm-agent/    │
                       │   tools/workspace-*  │
                       │   Anthropic SDK      │
                       └──────────────────────┘

┌──────────────┐           ┌──────────────┐
│    Redis     │           │   Adminer    │
│  Cache :6379 │           │  DB UI :8080 │
└──────────────┘           └──────────────┘
```

02. Aplicaciones

W apps/website/ — Portal Publico

ASPECTO	DETALLE
Stack	Vanilla JS + Tailwind CSS v3 + Vite v5 + PostCSS + Playwright
Proposito	Buscador de propiedades UAE para hispanohablantes
Consume	API REST de `apps/api/`

Paginas (HTML + JS pareados)

PAGINA	HTML	JS
Homepage / Landing	`index.html`	`src/main.js`
Listado propiedades v1	`propiedades.html`	`src/propiedades.js`
Listado propiedades v2	`propiedades-v2.html`	`src/propiedades-v2.js`
Detalle propiedad	`propiedad.html`	`src/propiedad.js`
Listado developers	`desarrolladores.html`	`src/desarrolladores.js`
Detalle developer	`desarrollador.html`	`src/desarrollador.js`
Blog	`blog.html`	`src/blog.js`
Articulo	`articulo.html`	`src/articulo.js`

A apps/api/ — Backend REST

ASPECTO	DETALLE
Stack	Node.js (ESM) + Express 4 + PostgreSQL + Redis
Puerto	3001
Rutas	`/inbox` (PM Agent chat), `/properties` (busqueda/filtros)
Entry	`src/index.js`

D apps/dashboard/ — Centro de Operaciones

ASPECTO	DETALLE
Stack	React 19 + React Router v7 + Recharts + Vite v7 (front) / Express 5 (back)
Puerto	5173 (front) + 3001 (back)
Backend	`server.js` (87KB monolitico) — importa `tools/pm-agent/`
i18n	Custom `LanguageContext.jsx` + `translations.js`

Componentes React

`BrainstormPanel`, `DailyAiSummary`, `DailyCoverageAlert`, `DailyEodModal`, `DailyTrends`, `DepartmentKanban`, `InboxPanel`, `Layout`, `PMAgentChat`, `PipelineBoard`, `WeeklyReport`

Rutas del Dashboard

RUTA	VISTA
`/`	Board de proyectos + PM Chat
`/workspace`	Vista de todos los agentes
`/workspace/:deptId`	Departamento
`/workspace/:deptId/weekly`	Planning semanal
`/workspace/:deptId/daily`	Standup diario
`/workflows`	Hub de workflows WAT
`/skills`	Analytics de skills
`/inbox`	Gestion de ideas/propuestas
`/pm-reports`	Archivo de reportes PM
`/workspace/audit`	Log de auditoria
`/workspace/intelligence`	Hub de investigacion

03. Tools y Scripts

Mas de **30 scripts Node.js** organizados en 7 modulos. Todos deben incluir `trackSkill()` al inicio.

```
tools/
├── Scrapers & Utilidades
│   ├── apify_propertyfinder.js       → PropertyFinder.ae via Apify
│   ├── apify_panicselling.js         → Price drops panicselling.xyz
│   ├── fetch_dataset.js              → Descarga datasets Apify
│   ├── check_db_count.js             → Diagnostico: filas por tabla
│   └── generate-business-plan-pdf.js → PDF del Business Plan
│
├── db/                               → Base de datos (15 archivos)
│   ├── pool.js                       → Conexion PostgreSQL compartida
│   ├── schema.sql + seed.sql         → Esquema + datos iniciales
│   ├── memory.js                     → Memoria agente (key-value)
│   ├── wat-memory.js                 → Memoria cross-agente
│   ├── save_project.js               → Guardar proyectos PM
│   ├── detect_duplicates.js          → Duplicados cross-broker
│   └── 4 migraciones SQL
│
├── pm-agent/                         → Cerebro del PM Agent
│   ├── core.js                       → Chat AI + generacion proyectos
│   ├── context-builder.js            → Contexto rico desde DB
│   ├── context-auditor.js            → Auditoria de completitud
│   └── inbox-cli.js                  → CLI inbox
│
├── research-agent/                   → Intelligence externa
│   ├── orchestrator.js               → Orquestador principal
│   └── 3 fetchers + filtro relevancia
│
├── telegram/                         → Bot Telegram (PM Agent movil)
├── translate/                        → Traduccion inmobiliaria EN→ES
└── workspace-skills/                 → Tracking y reportes
    ├── skill-tracker.js              → trackSkill() obligatorio
    ├── eod-generator.js              → Reportes end-of-day
    └── weekly-generator.js           → Resumenes semanales
```

04. Base de Datos

PostgreSQL 16 con 17 tablas, indices GiST para geolocalizacion y campos JSONB para datos heterogeneos.

TABLA	PROPOSITO	CAMPOS CLAVE
`properties`	Listings inmobiliarios de UAE	pf_id (PK), location, pricing, features (JSONB), images (JSONB), GiST(lat,lng)
`price_drops`	Price drops de lujo	original_price, current_price, drop_percentage
`projects`	Proyectos del PM Agent	name, problem, solution, status, success_metrics (JSONB)
`phases`	Fases de proyecto	project_id (FK), name, order
`tasks`	Tareas por fase	phase_id (FK), agent, effort (S/M/L), status, priority
`agents`	Registro de agentes	id, name, role, department, skills (JSONB), tools (JSONB)
`agent_memory`	Memoria key-value por agente	agent_id, key, value, scope (shared/private)
`inbox_items`	Ideas y propuestas	status: chat → borrador → proyecto → discarded
`pm_reports`	Reportes estrategicos	type, content, agent_id
`raw_events`	Event stream granular	event_type (tool_call, message, error, commit)
`eod_reports`	Reportes diarios	agent_id, date, content
`audit_log`	Auditoria del sistema	action, entity, details
`weekly_sessions`	Sesiones semanales	department, week, status
`weekly_brainstorms`	Brainstorms por agente	session_id, agent_id, ideas
`workflow_runs`	Ejecucion de workflows	workflow, status, duration, output

05. Infraestructura Docker

5 servicios orquestados via `docker-compose.yml`:

SERVICIO	IMAGEN	PUERTO	DESCRIPCION
postgres	postgres:16-alpine	5433	DB primaria. Auto-carga schema.sql + seed.sql
redis	redis:7-alpine	6379	Capa de cache para API
api	Dockerfile local	3001	Backend REST Express
dashboard	Dockerfile local	5173	Frontend React + backend Express
adminer	adminer:latest	8080	UI de administracion de DB

Dependencias compartidas (package.json raiz)

DEPENDENCIA	VERSION	USO
`@anthropic-ai/sdk`	^0.78.0	Claude API (PM Agent, Dashboard, traduccion)
`pg`	^8.13.0	PostgreSQL client
`playwright`	^1.58.2	Headless browser (screenshots, PDFs)
`telegraf`	^4.16.3	Telegram bot framework
`axios`	^1.7.0	HTTP client
`openai`	^6.25.0	OpenAI SDK (opcion secundaria)
`dotenv`	^16.4.0	Variables de entorno

06. Framework WAT

El framework **WAT (Workflows · Agents · Tools)** organiza toda la inteligencia del sistema en capas declarativas.

```
.claude/
├── CLAUDE.md                 ← Registro maestro (cargado SIEMPRE)
├── BUSINESS_PLAN.md          ← Norte estrategico
│
├── agents/ (9 agentes en 6 departamentos)
│   ├── product/    → pm-agent
│   ├── data/       → data-agent
│   ├── dev/        → dev-agent
│   ├── design/     → frontend-agent
│   ├── content/    → content-agent, translation-agent
│   ├── marketing/  → marketing-agent, research-agent
│   └── ops/        → wat-auditor-agent
│
├── skills/ (39 skills en 8 dominios)
│   ├── ops/        → 7 skills
│   ├── content/    → 1 skill
│   ├── design/     → 2 skills
│   ├── data/       → 7 skills
│   ├── ejecucion/  → 8 skills
│   ├── gtm/        → 5 skills
│   ├── marketing/  → 4 skills
│   └── producto/   → 5 skills
│
├── workflows/  → 7 SOPs cross-agente
├── hooks/      → 3 lifecycle hooks
└── rules/      → 3 reglas auto-activadas
```

Agentes del Sistema

AGENTE	DEPARTAMENTO	SKILLS	ROL
PM Agent	product/	21	Coordina todos los agentes, sprints, backlog, proyectos
Data Agent	data/	3	Extrae, limpia y normaliza datos de propiedades UAE
Frontend Agent	design/	3	UI/UX, creatividades, implementacion visual
Marketing Agent	marketing/	6	Campanas, copies, metricas, posicionamiento
Content Agent	content/	1	Fichas de propiedades, blog, SEO en espanol
Translation Agent	content/	1	Arabe/ingles → espanol con precision inmobiliaria
Dev Agent	dev/	1	Features, bugs, PRs en el codebase
Research Agent	ops/	1	Monitorea fuentes externas, genera intelligence
WAT Auditor	ops/	1	Auditoria del sistema WAT: consistencia y gaps

07. Interconexiones entre Documentos

El sistema WAT tiene **6 capas** de documentos que se referencian mutuamente:

```
╔═══════════════════════════════════════════════════════════╗
║  CAPA 0: FUNDACION (cargados SIEMPRE)                     ║
║  CLAUDE.md → registra agentes, skills, memoria, convenciones ║
║  BUSINESS_PLAN.md → norte estrategico                     ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 1: REGLAS (auto-activadas por contexto)             ║
║  business-plan-alignment → lee BUSINESS_PLAN, flag decisions ║
║  brand-guidelines → constrains toda UI, leido por frontend   ║
║  auto-dev-server → restart automatico en cambios          ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 2: AGENTES (9 .md con skills y tools asignados)       ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 3: SKILLS (39 invocables con /comando)              ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 4: WORKFLOWS (7 SOPs que encadenan skills)          ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 5: MEMORY BUS (agent_memory, shared/private)        ║
╟═══════════════════════════════════════════════════════════╢
║  CAPA 6: HOOKS (protect-files, memory-enforcement, reinject) ║
╚═══════════════════════════════════════════════════════════╝
```

Grafo de Dependencias entre Agentes

```
                    ┌──────────────┐
                    │   PM Agent   │  ← Coordina todo
                    │  21 skills   │
                    │  3 workflows │
                    └──────┬───────┘
            ┌──────────────┼──────────────┐
            ▼              ▼              ▼
┌────────────┐  ┌──────────────┐  ┌──────────────┐
│ Data Agent │  │Marketing     │  │Content Agent │
│ 3 skills   │  │Agent 6 skills│  │ 1 skill      │
└────────┬───┘  └──────┬───────┘  └──────┬───────┘
         │     ┌───────┘                 │
         ▼     ▼                         ▼
┌─────────────┐                  ┌─────────────┐
│Frontend Agent│ ←───────────────│Translation  │
│ 3 skills    │  recibe copy     │Agent 1 skill │
└─────────────┘                  └─────────────┘

┌─────────────┐     ┌─────────────┐
│Research Agent│──→│WAT Auditor  │
│ 1 skill     │feeds│ 1 skill     │
└─────────────┘intel└─────────────┘
```

08. Cadenas de Skills y Workflows

Los skills se **sugieren mutuamente**, creando cadenas naturales de ejecucion. Los workflows las formalizan con checkpoints de usuario.

Cadena de Estrategia de Producto

```
/estrategia-producto ──► /analisis-competidores ──► /battlecard-competitivo
        │                        │
        └──► /tamanio-mercado ◄──┘
                  │
                  └──► /segmento-entrada
```

Cadena GTM (Go-to-Market)

```
/segmento-entrada ──► /estrategia-gtm ──► /loops-crecimiento
(escribe beachhead     (lee beachhead          |
 en memoria)            de memoria)    /ideas-marketing
```

Cadena de Ejecucion

```
/crear-prd ──► /priorizar-features ──► /planificar-sprint
    │
    ├──► /pre-mortem (riesgos)
    └──► /historias-usuario (detalle)

/pm-challenge ──► (crea proyecto) ──► /cerrar-proyecto
```

Cadena de Inteligencia

```
/research-monitor ──► escribe latest_research_report
                            |
                      /wat-audit (lee report en Paso 2e)
                            |
                      /pm-context-audit (verifica alineacion)
```

Workflows (7 SOPs)

WORKFLOW	AGENTE LIDER	SKILLS ENCADENADOS	FRECUENCIA
sprint-planning	PM Agent	/priorizar-features → /planificar-sprint	Cada lunes
gtm-planning	PM + Marketing	/segmento-entrada → /analisis-competidores → /ideas-posicionamiento → /estrategia-gtm	Inicial + trimestral
marketing-planning	Marketing Agent	/metricas-norte → /mapa-viaje-cliente → /ideas-marketing → /ideas-posicionamiento	Mensual

WORKFLOW	AGENTE LIDER	SKILLS ENCADENADOS	FRECUENCIA
data-intelligence	Data Agent	/consultas-sql → /analisis-cohortes	Semanal / mensual
scrape_propertyfinder	Data Agent	/propertyfinder-scraper (via Apify)	Bajo demanda
screenshot_design_loop	Frontend Agent	/ui-ux-pro-max → /screenshot-loop (iterativo)	Por tarea UI
pm-review	PM Agent	/pm-challenge → save_project.js	Continuo

09. Flujos de Datos End-to-End

Flujo: Nueva Idea → Proyecto en Produccion

- **Idea ingresa** via Telegram Bot, Dashboard UI o agentes → `inbox_items` (status: chat)
- **PM Agent procesa** con `/pm-challenge` → lee `pm-agent.md` + `BUSINESS_PLAN.md` → activa regla `business-plan-alignment`
- **Borrador creado** → `inbox_items` (status: borrador) → usuario revisa y aprueba
- **Proyecto creado** → `projects` + `phases` + `tasks` en PostgreSQL
- **Sprint planificado** → `/planificar-sprint` asigna tasks a agentes
- **Agentes ejecutan** → leen memoria, ejecutan tools, escriben en memoria, `trackSkill()`
- **EOD generado** → `eod-generator.js` → `eod_reports` → Dashboard
- **Proyecto cerrado** → `/cerrar-proyecto` con summary y audit log

Flujo: Scraping → Propiedad en la Web

- `/propertyfinder-scraper` → verifica `last_scrape_at` en WAT Memory
- `apify_propertyfinder.js` → llama Apify API → recibe dataset
- Normaliza y guarda en `properties` (upsert por `pf_id`)
- `/detectar-duplicados` → marca `duplicate_of` (nunca borra)
- `apps/api/` sirve via REST `/properties` con filtros
- `apps/website/` consume y muestra al usuario hispanohablante

Flujo: Auditoria Semanal del Sistema

- `/research-monitor` → monitorea Anthropic docs, GitHub releases, Reddit
- Escribe `latest_research_report` en `agent_memory` (shared)
- `/wat-audit` → lee research report via `!backtick` injection → audita TODO .claude/
- Produce reporte con score (100 - critical*10 - warning*3 - suggestion*1)
- `/pm-context-audit` → lee `last_audit_score` → valida contra BUSINESS_PLAN

Sistema de Memoria (Memory Bus)

```
┌──────────────────────────────────────────────────────────┐
│                  MEMORY BUS (agent_memory table)          │
│                                                           │
│  Escritura: memory.js set <agent> <key> <value> [shared]  │
│  Lectura propia: memory.js get <agent> <key>              │
│  Lectura cross: wat-memory.js check <agent> <key>         │
│  Vista global: wat-memory.js status                       │
│                                                           │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐     │
│  │PM Agent  │ │Data Agent│ │Marketing │ │Research  │     │
│  │last_sprint│ │last_scrape│ │north_star│ │latest_   │     │
```

```
│  │icp_count │ │total_props│ │beachhead │ │research_ │        │
│  │audit_score│ │last_dedup│ │channels  │ │report    │        │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘        │
│         Skills leen via !backtick injection al cargarse       │
└──────────────────────────────────────────────────────────────┘
```

10. Decisiones de Arquitectura

Cada decision tecnologica tiene un **por que** alineado con la etapa actual del proyecto: MVP rapido, iteracion constante, y preparacion para escalar.

VANILLA JS EN WEBSITE (NO REACT)

Rendimiento maximo en carga inicial, mejor SEO sin SSR, y simplicidad para un MVP donde las paginas son mayormente estaticas con datos dinamicos. React se reserva para el Dashboard donde la interactividad justifica el overhead.

EXPRESS MONOLITICO EN DASHBOARD

Velocidad de desarrollo e iteracion rapida. Un solo archivo server.js (87KB) permite pivotar rapidamente sin overhead de microservicios. Se descompondra cuando el volumen de endpoints lo justifique.

POSTGRESQL + JSONB

Flexibilidad schema-on-read para datos inmobiliarios heterogeneos (cada broker estructura diferente). JSONB permite queries SQL sobre campos dinamicos sin migraciones constantes. GiST para busquedas geoespaciales.

PLAYWRIGHT (NO PUPPETEER)

Ya en el stack para screenshots del design loop. API mas moderna y estable que Puppeteer. Un solo browser engine para PDFs, screenshots y testing. Evita duplicar dependencias.

AGENTES COMO ARCHIVOS .MD

Configuracion declarativa, auditable con git, y versionable. Cada agente es un .md que define su rol, skills, tools y memoria. El WAT Auditor puede leerlos y detectar inconsistencias automaticamente.

MEMORY BUS EN POSTGRESQL

Persistencia entre sesiones, queries SQL para correlacion de estado, y coordinacion cross-agente sin servicios adicionales. La tabla agent_memory con scope shared/private es simple pero suficiente para el estado actual.

SKILLS 2.0 CON FRONTMATTER

Control granular de costos (haiku/sonnet/opus por skill), aislamiento de contexto (context:fork), y restriccion de tools por skill. Permite optimizar el gasto en API sin sacrificar calidad donde importa.

APIFY PARA SCRAPING

Infraestructura managed que evita bloqueos de IP, gestiona proxies, y escala automaticamente. El costo por run es predecible. Evita mantener infraestructura propia de scraping que es costosa y fragil.

TELEGRAM BOT COMO INTERFAZ

Acceso movil al PM Agent con friccion cero. El usuario puede enviar ideas desde cualquier lugar sin abrir el Dashboard. Telegraf es ligero y el bot actua como puente directo al inbox pipeline.

DOCKER COMPOSE (NO KUBERNETES)

Complejidad adecuada para la etapa actual. 5 servicios no justifican un orquestador. Docker Compose permite levantar todo el stack con un comando. Kubernetes llegara cuando haya multi-region o auto-scaling real.

11. Mejoras Futuras para Escalar

Roadmap tecnico de mejoras ordenadas por **prioridad e impacto** para cuando Emiralia crezca en usuarios, datos y mercados.

ALTA

CI/CD Pipeline

GitHub Actions para deploy automatico, tests, linting y validacion de schema. Elimina deploys manuales y reduce errores en produccion. Prerequisito para escalar el equipo.

ALTA

Autenticacion (Auth System)

Supabase Auth o Auth0 para usuarios finales (inversores) y developers (clientes B2B). Roles, permisos, y acceso a datos segregado por developer. Prerequisito para monetizacion.

ALTA

Search Engine (Meilisearch)

Busqueda full-text de propiedades en espanol con facets, filtros instantaneos y typo-tolerance. Reemplaza queries SQL directas que no escalan con volumen. UX critica para el portal publico.

ALTA

CDN para Imagenes

Cloudflare R2 o S3 + CloudFront para servir fotos de propiedades desde edge. Reduce latencia para usuarios en Espana y LatAm. Transformaciones on-the-fly (resize, webp) para performance.

MEDIA

Descomponer server.js Monolitico

Separar el server.js de 87KB del Dashboard en modulos: rutas API, middleware, servicios. No microservicios completos aun, pero si modularizacion para mantenibilidad y testing.

MEDIA

API Gateway

Centralizar autenticacion, rate limiting, CORS y routing entre servicios. Punto unico de entrada para website, dashboard y futuros clientes API (developers B2B).

MEDIA

Message Queue (Redis Streams)

Desacoplar scrapers y procesamiento asincrono. Los scrapes de PropertyFinder generan eventos que se procesan independientemente (dedup, normalizacion, notificaciones). Ya tenemos Redis.

MEDIA

Monitoring (Sentry + Grafana)

Observabilidad de errores, performance y metricas de negocio. Alertas automaticas cuando un scraper falla, el API tiene latencia alta, o las propiedades dejan de actualizarse.

MEDIA

Caching Avanzado (Redis)

Cache de queries frecuentes de propiedades, resultados de busqueda, y datos de developers. Invalidacion inteligente cuando llegan nuevos datos de scraping. Ya tenemos Redis desplegado.

MEDIA

WebSockets (Notificaciones Real-time)

Notificaciones push cuando hay nuevas propiedades, price drops, o actualizaciones de proyectos. Mejora la retencion y engagement de inversores activos.

BAJA

Multi-region DB

Replicas read-only de PostgreSQL en Europa para baja latencia en Espana/LatAm. Solo cuando el volumen de usuarios justifique el costo de infraestructura multi-region.

BAJA

Mobile App (React Native)

Aplicacion nativa para inversores hispanohablantes. Push notifications de oportunidades, calculadora ROI offline, y acceso rapido a favoritos. Complementa al portal web, no lo reemplaza.

12. Patrones Arquitectonicos

Los 7 patrones que definen como funciona el sistema Emiralia:

PATRON	DESCRIPCION
Memory Bus	Todos los agentes escriben en `agent_memory` (PostgreSQL) con scope shared. Skills leen al cargarse via `!backtick` injection. Estado visible system-wide sin hardcoding.
Research → Audit Chain	Research Agent monitorea fuentes externas semanalmente → escribe `latest_research_report` → WAT Auditor lo lee antes de cada auditoria → cross-referencia en Check 2e.
Skill Cascade	Skills se sugieren mutuamente (`/crear-prd` → `/priorizar-features` → `/planificar-sprint`), creando secuencias naturales sin workflows formales.
Workflow Orchestration	Workflows encadenan 2-4 skills con checkpoints obligatorios de usuario entre cada paso. Escritura cross-agente en memoria compartida al finalizar.
Skills 2.0 Tiers	`haiku` para tracking y queries ligeras, `sonnet` para auditorias y analisis, `opus` para PRDs estrategicos. Optimiza costo/calidad por skill.
disable-model-invocation	Skills con side effects solo se invocan con `/comando` explicito del usuario. Previene ejecucion accidental de operaciones costosas o irreversibles.
Shared Audit Scoring	`/wat-audit` y `/pm-context-audit` usan formula identica: `100 - critical*10 - warning*3 - suggestion*1` . Scores comparables y visibles system-wide.